

MÁSTER EN ROBÓTICA Y AUTOMATIZACIÓN
LABORATORIO DE ROBÓTICA

Práctica de laboratorio – NAV2

Realizado por:

Glen Alejandro Guerrero Burbano

Profesor:

Daniel Vicente Rodrigo Muñoz

30 de mayo de 2026

I. RESUMEN

Este informe documenta la segunda fase del experimento de navegación con un robot móvil TurtleBot3 equipado con LiDAR, sobre ROS 2 Jazzy y el simulador Gazebo. Partiendo del escenario simulado desarrollado previamente, el trabajo se organiza en dos etapas claramente diferenciadas. En la primera, el robot se conduce manualmente mediante teleoperación por teclado para recorrer el entorno y construir un mapa con la técnica de mapeo y localización simultáneos (SLAM); el mapa resultante se guarda en disco. En la segunda etapa, ese mapa se reutiliza como entrada fija para la navegación autónoma: el robot se localiza dentro del mapa conocido mediante el algoritmo AMCL y, a partir de un punto objetivo, la pila Nav2 realiza la planificación de la trayectoria y el control del movimiento hasta alcanzarlo, evitando los obstáculos.

II. MATERIALES Y MÉTODOS:

Se utilizó una máquina virtual con sistema operativo Ubuntu 24.04 LTS, sobre la que se instaló la distribución ROS 2 Jazzy Jalisco. Como simulador se empleó la versión moderna de Gazebo, que sustituye al antiguo Gazebo Classic a partir de esta distribución. La pila de navegación utilizada fue Nav2, junto con los paquetes mínimos de simulación del TurtleBot3 que se instalan de forma conjunta. El robot seleccionado fue el modelo TurtleBot3 Waffle, que incorpora un sensor LiDAR rotativo como sensor principal de percepción. Se creó un paquete propio *“mi_paquete_nav”* y el escenario simulado desarrollado en Fusion 360, que actúan como base sobre la que se incorpora ahora la navegación autónoma. Para la teleoperación se empleó adicionalmente el paquete *teleop_twist_keyboard*.

PARTE 1 – Estructura del nuevo paquete:

El escenario se construyó a partir de un modelo tridimensional en formato STL con Fusion 360, encapsulado en un modelo SDF con geometría de visualización y de colisión, e incluido de forma modular en el mundo simulado. El entorno se hizo de aproximadamente 5x5 metros con unos obstáculos para poder verificar el comportamiento de SLAM

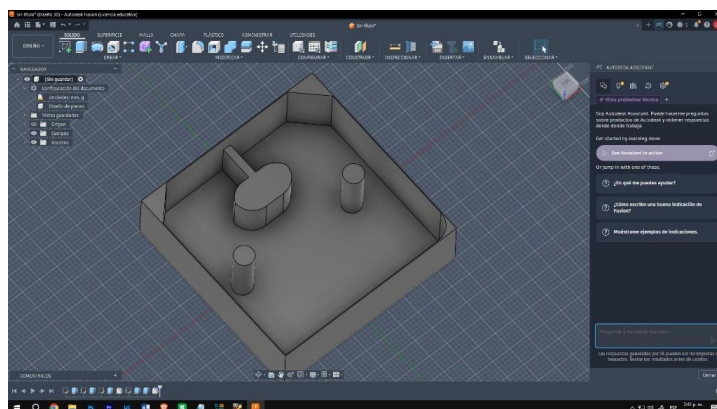


Figura 1. STL

La percepción se apoya en el LiDAR del robot, cuya señal alimenta el proceso de mapeo y planificación. Dado que el escenario carece de mapa previo, la navegación se ejecutó en modo SLAM, de modo que el mapa se construye en tiempo real a medida que el robot recorre el entorno. Los resultados muestran que el robot alcanza el punto objetivo planificando trayectorias y evitando los obstáculos del escenario, con un árbol TF consistente y temporización coherente bajo tiempo simulado, validando así la correcta integración del sistema.

```

mi_paquete_nav/
├── CMakeLists.txt
├── package.xml
├── worlds/
│   └── escenario.sdf      # Mundo: iluminación + inclusión del escenario
├── models/
│   └── escenario/
│       ├── model.config
│       ├── model.sdf      # Encapsula el STL (visual + colisión)
│       └── meshes/
│           └── escenario.stl # Modelo 3D del escenario creado en fusión 360
├── launch/
│   └── escenario_nav.launch.py # Punto único de entrada
├── params/
│   └── nav2_params.yaml      # Parámetros de navegación
└── maps/
    └── mi_mapa.pgm / .yaml    # Mapa resultante del SLAM

```

El archivo `escenario.sdf` define el mundo simulado, incluyendo la iluminación y la inclusión modular del escenario mediante una etiqueta de inclusión con su pose definida de forma explícita. El archivo `model.sdf` encapsula la malla STL del escenario, declarándola tanto en su geometría de visualización como en la de colisión. El archivo `escenario_nav.launch.py` constituye el punto único de entrada que coordina todo el arranque. Finalmente, `nav2_params.yaml` contiene los parámetros de configuración de la navegación.

PARTE 3 – Construcción del escenario en Gazebo: El escenario de navegación se construyó a partir del modelo anterior en STL. Dado que un archivo STL no almacena información sobre unidades ni materiales, se prestó especial atención a la escala del modelo y a su correcta definición de colisiones. El modelo se encapsuló en un archivo SDF declarando la misma malla en la geometría de visualización y en la geometría de colisión, ya que es esta última la que determina cómo el sensor LiDAR percibe las paredes y los obstáculos del entorno. El escenario se declaró como elemento estático para que permanezca fijo durante toda la simulación y no se vea afectado por la física.

El mundo se estructuró de forma modular: el modelo del escenario se define como un modelo independiente y se incorpora al mundo mediante una inclusión con su pose especificada de forma explícita, lo que facilita el mantenimiento y la repetibilidad del montaje. La superficie del escenario se alineó de manera que su cara superior coincidiera

con el plano de referencia del mundo, garantizando que el robot pudiera insertarse posteriormente apoyado sobre el suelo sin interpenetración. En esta fase se verificó visualmente en Gazebo que el escenario presentaba una escala coherente, que disponía de geometría de colisión y que se mantenía estable en reposo.

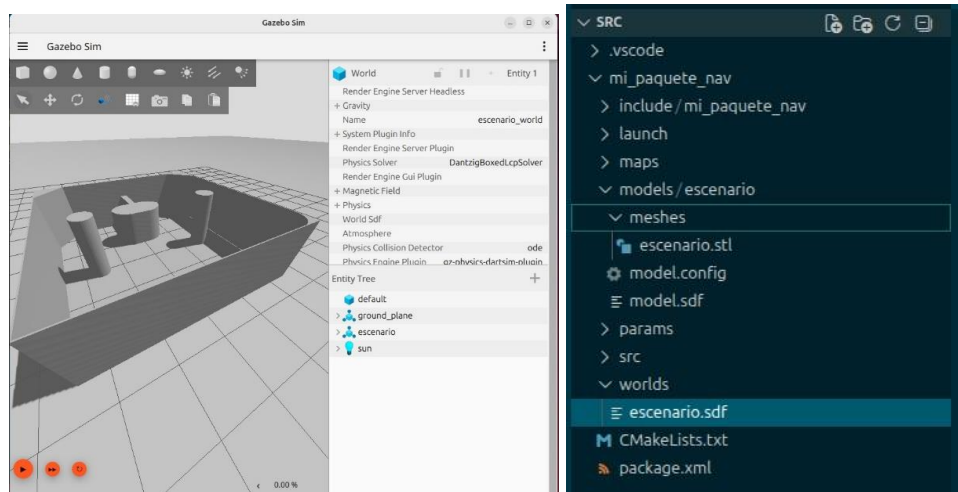


Figura 2. Gazebo STL

PARTE 3 – Launch Simulación - MAPEO:

El archivo de lanzamiento `escenario_nav.launch.py` se diseñó como punto único de entrada del experimento. Su función es coordinar, en una sola ejecución, el arranque del mundo en Gazebo, la inserción del robot en una pose inicial no conflictiva, la activación del uso de tiempo simulado y el arranque de los nodos de navegación. Para que el simulador localice correctamente la malla del escenario, el lanzamiento configura la variable de entorno de rutas de recursos de Gazebo apuntando a la carpeta de modelos del paquete. De este modo, mundo, robot y nodos quedan alineados desde el inicio en nombres, tiempo y marcos, evitando configuraciones manuales dispersas.

La primera etapa consiste en construir el mapa del escenario conduciendo el robot manualmente. Para ello se arranca la simulación en modo SLAM, de modo que el sistema construye el mapa a partir de las lecturas del LiDAR a medida que el robot se desplaza. El control del movimiento se realiza desde el teclado mediante un nodo de teleoperación que publica comandos de velocidad. La elección de conducir el robot manualmente, en lugar de dejar que la navegación lo haga sola, permite recorrer el entorno de forma ordenada y completa, obteniendo un mapa más limpio y con mejor cobertura.

El sistema se arranca con el lanzamiento de navegación en modo SLAM y, en una terminal independiente, el nodo de teleoperación por teclado. La ventana del nodo de teleoperación debe mantener el foco para capturar las pulsaciones de teclas, que controlan el avance, retroceso y giro del robot.

Terminal 1: simulación + SLAM

`ros2 launch mi_paquete_nav escenario_nav.launch.py`

Terminal 2: teleoperación por teclado

```
ros2 run teleop_twist_keyboard teleop_twist_keyboard
```

Con el robot bajo control manual, se recorre la totalidad del escenario, prestando especial atención al perímetro y al contorno de cada obstáculo, mientras se observa en la interfaz de visualización cómo el mapa se va construyendo progresivamente. Una cobertura completa del entorno en esta etapa resulta determinante para la calidad de la localización posterior

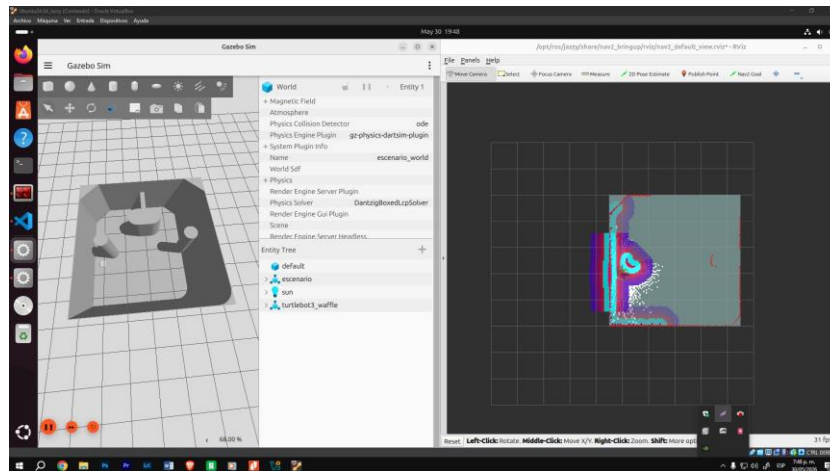


Figura 3. Teclado

Como se puede observar al no tener un mapa cargado o creado, se tiene SLAM puro y por eso se ve la posición inicial en Rviz del robot de esa forma, lo que queda en este punto es comenzar a hacer la navegación del robot para generar el mapa de ocupación

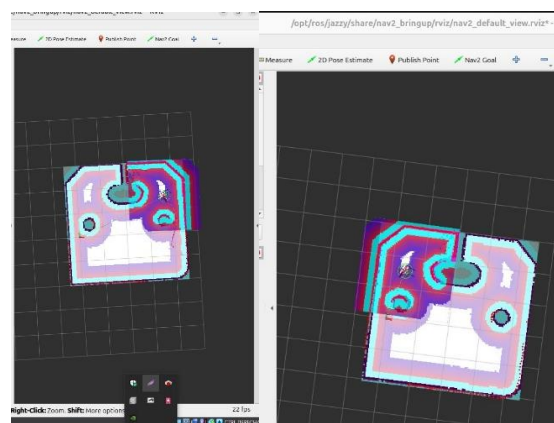


Figura 4. Mapa Rviz

Una vez recorrido por completo el escenario, el mapa construido se guarda en disco para poder reutilizarlo como entrada de la navegación autónoma. El proceso de guardado genera dos archivos complementarios: una imagen en escala de grises que representa la ocupación del entorno (espacio libre, obstáculos y zonas desconocidas) y un archivo de metadatos que define la resolución, el origen y los umbrales de ocupación. Ambos archivos se almacenaron en la carpeta de mapas del paquete propio.

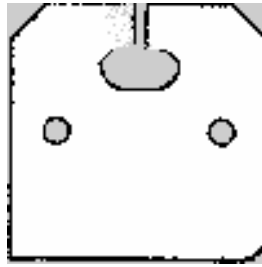


Figura 5. Mapa de ocupación.

PARTE 4 – Navegación autónoma

En la tercera etapa se reutiliza el mapa guardado para ejecutar la navegación autónoma. A diferencia del modo SLAM, donde el entorno se descubre sobre la marcha, aquí el mapa es fijo y conocido, y el robot debe determinar su propia posición dentro de él. Esta localización se realiza mediante el algoritmo AMCL, que compara de forma continua las lecturas del LiDAR con el mapa para estimar la pose del robot. Para ello se preparó un lanzamiento propio que carga el mapa y activa la localización en lugar del SLAM.

El arranque sigue siendo un punto único de entrada. Al iniciarse, la interfaz de visualización muestra el mapa cargado, y el primer paso consiste en proporcionar al sistema una estimación de la pose inicial del robot, indicando su posición y orientación aproximadas sobre el mapa. Este paso, que en el modo SLAM no era necesario, permite que la localización converja y sitúe correctamente al robot dentro del entorno conocido

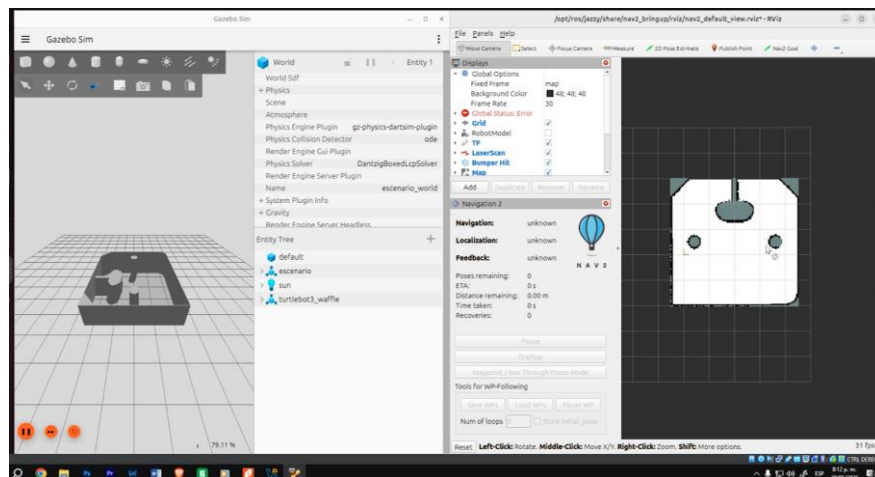


Figura 6. Launch

Una vez localizado el robot, se indica un punto objetivo dentro del mapa. A partir de ese momento, la planificación calcula una trayectoria global desde la posición actual hasta el objetivo, y el control genera de forma continua los comandos de velocidad necesarios para seguir dicha trayectoria, ajustándola en tiempo real ante los obstáculos detectados por el LiDAR. El robot se desplaza así de manera completamente autónoma, sin intervención del operador sobre su movimiento, lo que constituye la diferencia esencial respecto a la etapa de mapeo teleoperado.

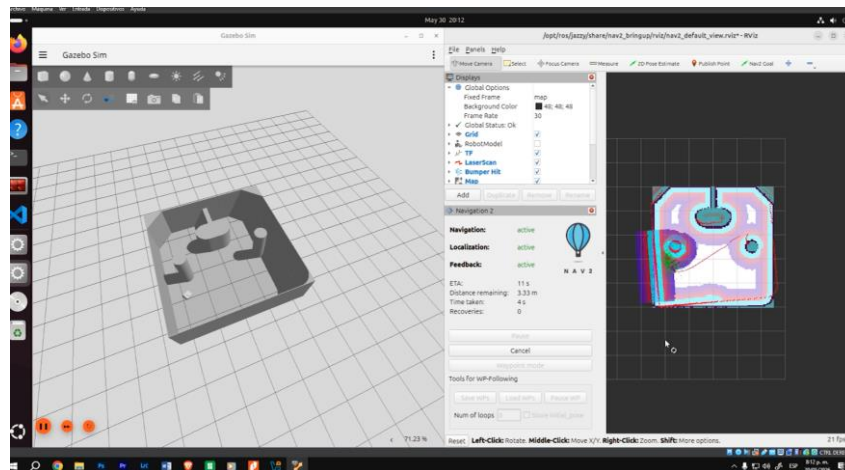


Figura 6. Navegación

Repositorio original git hub: <https://github.com/glenguerrero01/Control-robot-en-entorno-virtual/tree/main>

Video explicación: <https://youtu.be/GaFDvxMamDc>

III. CONCLUSIONES

El trabajo demuestra el ciclo completo de navegación autónoma de un robot móvil con LiDAR en simulación: desde la construcción de un mapa mediante conducción manual hasta el desplazamiento autónomo del robot sobre ese mapa. La separación del experimento en dos etapas permitió evidenciar con claridad la diferencia entre ambos paradigmas. En la etapa de mapeo, el sistema desconoce el entorno y es el operador quien dirige el robot mediante teleoperación por teclado para construir el mapa. En la etapa de navegación autónoma, el mapa es ya conocido y el robot se conduce por sí mismo, localizándose dentro del mapa y resolviendo de forma automática la planificación de la trayectoria y el control del movimiento.

La reutilización del mapa generado puso de manifiesto la importancia de una cobertura completa durante el mapeo: un mapa limpio y bien recorrido favorece que la localización converja con rapidez y precisión. Asimismo, se comprobó que el suministro de una estimación de la pose inicial es un paso necesario en la navegación con mapa conocido, a diferencia del mapeo en modo SLAM, ya que de él depende que el robot se sitúe correctamente en el entorno antes de comenzar a navegar.

En conjunto, el experimento valida la integración entre localización, planificación y control sobre la pila de navegación, empleando los planificadores y controladores que esta proporciona.

IV. REFERENCIAS

gz_ros2_control. (2024). ros-controls/gz_ros2_control. https://github.com/ros-controls/gz_ros2_control

Macenski, S., Foote, T., Gerkey, B., Lalancette, C., & Woodall, W. (2022). Robot Operating System 2: Design, architecture, and uses in the wild. Science Robotics, 7(66), eabm6074.

Open Navigation LLC. (2025). Nav2 Documentation – Getting Started. Recuperado de https://docs.nav2.org/getting_started/index.html

Open Robotics. (2024). ROS 2 Jazzy Jalisco Documentation. <https://docs.ros.org/en/jazzy/>

Open Robotics. (2024). Gazebo Harmonic Documentation. <https://gazebo.org/docs/harmonic/>